CHAPTER 4

Natural Deduction

In Chapter 3 you met judgments, inference rules and derivations, and used them to describe which programs of a small language are well typed. This chapter applies the same machinery to a different subject: logic itself. The question is old and simple. When is a proof a proof? Mathematicians reason by assuming things, drawing consequences, splitting into cases and discharging assumptions when they are no longer needed, and for most of history nobody wrote down the rules of that game precisely. In the mid-1930s Gerhard Gentzen did, and he called his system *natural deduction*, because it follows the way people actually reason.

Natural deduction matters to us for three reasons. It is the logic that Lean checks whenever you write a proof. Its rules obey a precise design principle, *harmony*, which explains their shapes and shows how one badly designed rule can wreck a logic. And its proofs turn out to be programs: the rules are, almost symbol for symbol, the typing rules of a functional language, and the way proofs simplify is the way programs run. Part II is built on that fact.

In this chapter you will learn:

- the difference between a proposition and a judgment, and what a hypothetical judgment $\Gamma \vdash A$ is;
- the introduction and elimination rules for $\wedge$, $\to$, $\vee$, $\top$ and $\bot$, with negation defined as $A \to \bot$, in tree notation and in sequent notation;
- harmony: why detours can be removed (local soundness) and why nothing is lost (local completeness), and how Prior's `tonk` shows that harmony matters;
- normalization, the subformula property and consistency, and a first look at the sequent calculus and cut elimination;
- how to prove propositional formulas in Lean, with proof terms and with the tactics `intro`, `exact`, `apply`, `constructor`, `cases`, `rcases`, `obtain`, `left`, `right`, `exfalso` and `contradiction`;
- how to define natural deduction *inside* Lean and prove it sound with respect to truth tables;
- how to build a proof checker in Rust that gives good error messages, and what happens to it when you add `tonk`.

4.1 Propositions and judgments

Formulas

We start with the objects that logic is about. Fix a supply of *atomic propositions*: statements whose internal structure we do not analyze, such as "the connection is open" or "the order was paid". Formulas are built from atoms with a handful of connectives:

$$A, B ::= P \mid \top \mid \bot \mid A \wedge B \mid A \vee B \mid A \to B$$

where P ranges over atoms and A, B over formulas (in rules, letters such as A, B, C stand for any formula), $\top$ ("truth") is a proposition that is trivially true, $\bot$ ("falsity", read "bottom") is one that is never true, and $\wedge$, $\vee$, $\to$ are "and", "or" and "implies". This is a grammar of trees, like the grammar of `Arith` in Chapter 3: $A \wedge B \to C$ is a tree whose root is $\to$. The usual conventions keep the parentheses down: $\neg$ binds tightest, then $\wedge$, then $\vee$, then $\to$, and $\to$ groups to the right, so $A \to B \to C$ means $A \to (B \to C)$.

Two connectives are deliberately missing, because we define them:

$$\neg A := A \to \bot \qquad\qquad A \leftrightarrow B := (A \to B) \wedge (B \to A)$$

"Not A" *means* "A implies falsity": to refute A is to show that assuming A leads to absurdity. This definition is not a shortcut. As you will see, it gives negation exactly the rules one would want, and Chapter 5 explains why it is the right reading of "not".

"A is true" is a judgment

The most useful distinction in this subject, stressed by Per Martin-Löf, is the one between a **proposition** and a **judgment**. A proposition, such as $A \wedge B$, is a piece of syntax, an object we talk about. A judgment is an act of knowledge *about* such an object. The judgment we care about here is

$$A\ true$$

read "A is true", or more precisely "we have evidence for A". There are other judgment forms. You have already seen "e has type τ" in Chapter 3, and "A is a well-formed formula" is another. Inference rules relate judgments, never propositions: the horizontal line in a rule is not a connective, and it says "if you know the judgments above, you may conclude the one below".

Hypothetical judgments

We rarely know that something is true outright. Mostly we know that something is true *provided* other things are: "if the payment cleared and the item is in stock, the order can ship". This is a **hypothetical judgment**, written

$$A_1, \ldots, A_n \vdash B$$

and read "assuming $A_1, \ldots, A_n$, B is true". The list on the left of the turnstile $\vdash$ is the **context**, usually called Γ, and the whole thing $\Gamma \vdash B$ is called a **sequent**. With an empty context, $\vdash B$ says that B is true outright.

> **WARNING**
>
> $A \vdash B$ and $\vdash A \to B$ are different judgments. The first is a hypothetical judgment about B; the second is a categorical judgment about a formula that happens to contain the symbol $\to$. The rules below make them *interderivable* (each follows from the other), but they live at different levels: $\vdash$ belongs to the metalanguage, $\to$ to the object language. Keeping them apart is what will let Chapter 6 ask "which rules govern the left side of $\vdash$?", a question about the judgment, not about any connective.

Is Γ a set, a multiset or a list? Choosing "set" quietly decides that assumptions can be reordered, ignored and reused at will. We take Γ to be a **list**, and in this chapter we use it as freely as ordinary logic does; Chapter 6 turns each of those freedoms into an explicit rule and asks what happens when one is removed, which is where Rust's ownership enters the story.

4.2 The rules of natural deduction

Gentzen organized the rules by connective. Each connective has **introduction rules**, which say how to prove a formula with that connective at the root, and **elimination rules**, which say how to use one. We write the rules in sequent style, with the context in every judgment.

Hypotheses

The simplest rule says that you may use what you assumed:

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \mathsf{Hyp}$$

The premise $A \in \Gamma$ is a side condition that you check by looking at the list, not a judgment you derive.

Conjunction

To prove $A \wedge B$, prove A and prove B. To use $A \wedge B$, take either part:

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\ \wedge\mathsf{I} \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\ \wedge\mathsf{E}_1 \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\ \wedge\mathsf{E}_2$$

Notice that both premises of $\wedge$I have the *same* context Γ. Whatever assumptions are available are available to both halves of the proof. This is called context *sharing*, and it is exactly the point where linear and affine logic (Part V) will part ways with ordinary logic.

Implication

To prove $A \to B$, assume A and prove B. To use $A \to B$, supply an A:

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\ \to\mathsf{I} \qquad \frac{\Gamma \vdash A \to B \qquad \Gamma \vdash A}{\Gamma \vdash B}\ \to\mathsf{E}$$

$\to$E is the ancient rule *modus ponens*. $\to$I is the interesting one: its premise has one more assumption than its conclusion. Reading upward, it says "to prove $A \to B$, add A to your assumptions and prove B". Reading downward, it says that once B is proved under the assumption A, the assumption can

be **discharged**: the conclusion $A \to B$ no longer depends on it. The two rules together explain the warning above: $\Gamma, A \vdash B$ holds exactly when $\Gamma \vdash A \to B$ does.

Two notations

Gentzen did not write contexts. In his **tree notation**, a derivation is a tree whose leaves are assumptions, and a rule that discharges an assumption marks the corresponding leaves with a label, here u:

$$\dfrac{\begin{array}{c}[A]^u \\ \vdots \\ B\end{array}}{A \to B}\ {\to}\mathsf{I}^u$$

The brackets say that the leaves A labeled u are closed by the $\to$I step with the same label; the dots stand for the derivation of B, which may use A any number of times. Sequent notation carries the same information by writing the open assumptions to the left of $\vdash$ at every step. Trees are compact on paper; sequents make the context explicit, which programs and the structural rules of Chapter 6 need. The Rust checker later in this chapter reads proofs in Gentzen's style and prints them in sequent style.

> **NOTE**
>
> Tree notation hides something. A single $\to$I step with label u closes *every* leaf labeled u, however many there are. Nothing in the notation counts the uses of an assumption, so using A twice is free. That silent freedom is the structural rule of *contraction*, and Chapter 6 brings it into the open.

Disjunction

To prove $A \vee B$, prove one of them. Using $A \vee B$ is reasoning by cases:

$$\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B}\ {\vee}\mathsf{I}_1 \qquad \dfrac{\Gamma \vdash B}{\Gamma \vdash A \vee B}\ {\vee}\mathsf{I}_2$$

$$\dfrac{\Gamma \vdash A \vee B \qquad \Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma \vdash C}\ {\vee}\mathsf{E}$$

$\vee$E says: if $A \vee B$ holds, and C follows from A, and C also follows from B, then C holds. Each case discharges its own assumption. The conclusion C is arbitrary: it is whatever you were trying to prove when you decided to split into cases. This rule looks very different from $\wedge$E, and we will see shortly that it has to.

Truth and falsity

$\top$ has one introduction rule with no premises, and no elimination rule. $\bot$ has no introduction rule, and one elimination rule:

$$\dfrac{}{\Gamma \vdash \top}\ \top\mathsf{I} \qquad \dfrac{\Gamma \vdash \bot}{\Gamma \vdash C}\ \bot\mathsf{E}$$

$\bot$E is *ex falso quodlibet*, "from falsity, anything", which you met in Chapter 1 as Rust's never type coercing to every type. It is the rule people find most suspicious, and harmony will explain why it is exactly right.

Negation, for free

Since $\neg A$ is $A \to \bot$, negation needs no rules of its own. The rules for $\to$ with $B := \bot$ read:

$$\frac{\Gamma, A \vdash \bot}{\Gamma \vdash \neg A}\,{\to}\mathsf{I} \qquad\qquad \frac{\Gamma \vdash \neg A \qquad \Gamma \vdash A}{\Gamma \vdash \bot}\,{\to}\mathsf{E}$$

To refute A, assume it and derive absurdity; a refutation of A together with a proof of A is absurd. Everything we prove about implication (harmony, normalization) therefore holds for negation automatically.

A first derivation

Let us derive $A \wedge B \vdash B \wedge A$. Write $\Gamma = A \wedge B$:

$$\dfrac{\dfrac{\dfrac{A \wedge B \in \Gamma}{\Gamma \vdash A \wedge B}\,\mathsf{Hyp}}{\Gamma \vdash B}\,{\wedge}\mathsf{E}_2 \qquad \dfrac{\dfrac{A \wedge B \in \Gamma}{\Gamma \vdash A \wedge B}\,\mathsf{Hyp}}{\Gamma \vdash A}\,{\wedge}\mathsf{E}_1}{\Gamma \vdash B \wedge A}\,{\wedge}\mathsf{I}$$

Read it from the bottom, the way you would search for it. The goal is a conjunction, so try ∧I, which asks for B and A separately. Neither is an assumption, but both are parts of the assumption $A \wedge B$, so ∧E2 and ∧E1 extract them, and Hyp closes each branch. One more step, ⟶I, discharges the assumption and gives $\vdash A \wedge B \to B \wedge A$.

The single assumption $A \wedge B$ was used twice, once in each branch, which is legal because ∧I shares its context. Remember this derivation: in Chapter 21 it becomes illegal, and in Rust, with a non-`Copy` pair taken apart in two places, it becomes error E0382, "use of moved value".

Here is a second example, with negation: $\vdash A \to \neg\neg A$, that is, $A \to ((A \to \bot) \to \bot)$. In tree notation:

$$\dfrac{\dfrac{\dfrac{[\neg A]^{v} \qquad [A]^{u}}{\bot}\,{\to}\mathsf{E}}{\neg\neg A}\,{\to}\mathsf{I}^{v}}{A \to \neg\neg A}\,{\to}\mathsf{I}^{u}$$

Applying the assumption $\neg A$ (label v) to the assumption A (label u) gives $\bot$; discharging v and then u gives the implication. Chapter 5 is largely about the fact that the converse, $\neg\neg A \to A$, has no derivation.

4.3 Proving it in Lean

Lean's logic is natural deduction, extended to a much richer language. Chapter 2 showed you `theorem`, `example` and the first tactics; this section shows how Lean represents propositions and proofs, and how each rule of natural deduction becomes either a function you can apply or a tactic. The code is in `code/lean/Book/Ch04/Tactics.lean`.

Propositions are types

In Lean a proposition is a type whose type is `Prop`:

```
/-- info: True : Prop -/
#guard_msgs in
#check True
```

```
/-- info: 2 + 2 = 4 : Prop -/
#guard_msgs in
#check 2 + 2 = 4

/-- info: 2 + 2 = 5 : Prop -/
#guard_msgs in
#check 2 + 2 = 5

/-- info: ∀ (p q : Prop), p ∧ q → q ∧ p : Prop -/
#guard_msgs in
#check ∀ (p q : Prop), p ∧ q → q ∧ p
```

`2 + 2 = 5` is a perfectly good proposition: false propositions are types too, just types with no elements. A **proof** of `p` is a term of type `p`, and checking a proof is type checking. This is the Curry–Howard correspondence of Chapter 10; for now, take it as Lean's way of implementing the judgment "A true": `h : p` means "`h` is evidence for `p`".

The rules are functions

Lean's core library provides the connectives `∧`, `∨`, `→`, `True`, `False` and `¬`, and their rules as named functions:

```
/-- info: @And.intro : ∀ {a b : Prop}, a → b → a ∧ b -/
#guard_msgs in
#check @And.intro

/-- info: @And.left : ∀ {a b : Prop}, a ∧ b → a -/
#guard_msgs in
#check @And.left

/-- info: @Or.inl : ∀ {a b : Prop}, a → a ∨ b -/
#guard_msgs in
#check @Or.inl

/-- info: @Or.elim : ∀ {a b c : Prop}, a ∨ b → (a → c) → (b → c) → c -/
#guard_msgs in
#check @Or.elim

/-- info: True.intro : True -/
#guard_msgs in
#check True.intro

/-- info: @False.elim : {C : Sort u_1} → False → C -/
#guard_msgs in
#check @False.elim
```

Read the types as rules. `And.intro` turns a proof of `a` and a proof of `b` into a proof of `a ∧ b`: that is ∧I, with `a` and `b` implicit (in braces), so that Lean infers them. `And.left` is ∧E1, `Or.inl` is ∨I1, `True.intro` is ⊤I, and `Or.elim` is ∨E: a disjunction, a way from `a` to `c`, a way from `b` to `c`, hence `c`. `False.elim` is ⊥E; its result can be any `Sort`, even a type of data, like Rust's never type coercing to anything. There is no function for →I or →E: implication *is* the function type, so →I is writing a `fun` and →E is applying it. Negation is defined exactly as in the text:

```
/--
info: @[implicit_reducible] def Not : Prop → Prop :=
fun a => a → False
-/
#guard_msgs in
#print Not
```

(The attribute `implicit_reducible` tells Lean to unfold `Not` freely when it compares types, so that a proof of `p → False` is accepted where `¬p` is expected.)

Proof terms

With these functions, a proof is an expression:

```
/-- `∧E₂` and `∧E₁` feed `∧I`: the derivation of `A ∧ B ⊢ B ∧ A`. -/
theorem and_swap (p q : Prop) (h : p ∧ q) : q ∧ p :=
  And.intro (And.right h) (And.left h)

/-- `→I` is a `fun`, `→E` is application. -/
theorem imp_trans (p q r : Prop) : (p → q) → (q → r) → (p → r) :=
  fun hpq hqr hp => hqr (hpq hp)

/-- `∨E`: one branch for each way the disjunction could have been proved. -/
theorem or_swap (p q : Prop) (h : p ∨ q) : q ∨ p :=
  Or.elim h (fun hp => Or.inr hp) (fun hq => Or.inl hq)

/-- `⊥E`: from a proof of `False`, anything. -/
theorem explode (p : Prop) (h : False) : p :=
  False.elim h
```

The first is the derivation of $A \wedge B \vdash B \wedge A$ as a term: the hypothesis `h` plays the role of Γ, and the nesting of `And.intro`, `And.right` and `And.left` is the shape of the tree. In `imp_trans`, `fun hpq hqr hp => ...` is three uses of $\to$I and `hqr (hpq hp)` two uses of $\to$E. Lean also offers shorthand: angle brackets for introductions, `.1` and `.2` for projections, and patterns in `fun`:

```
/-- The same proofs with Lean's shorthand: `⟨_, _⟩` builds a pair, `.1` and
`.2` project, and `fun ⟨hp, hq⟩ => ...` takes the pair apart. -/
theorem and_swap' (p q : Prop) : p ∧ q → q ∧ p :=
  fun ⟨hp, hq⟩ => ⟨hq, hp⟩

theorem and_swap'' (p q : Prop) (h : p ∧ q) : q ∧ p :=
  ⟨h.2, h.1⟩
```

Tactics: the rules read upward

For longer proofs it is easier to work the way you searched for the derivation above: from the goal upward. That is **tactic mode**. After `by`, Lean shows a *goal*, a sequent `Γ ⊢ A` listing the hypotheses you have and what you must prove, and each tactic applies a rule backwards, replacing the goal by the premises of that rule. Here is each rule as a tactic.

`intro` is $\to$I and `exact` closes a goal with a term of exactly its type (here rule Hyp):

```
/-- `intro` is rule →I, read bottom-up: to prove `p → q`, assume `p` (the
assumption gets the name `hp`) and prove `q`. `exact` closes the goal with a
term of exactly the goal's type, here the assumption itself (rule Hyp). -/
theorem identity (p : Prop) : p → p := by
  intro hp      -- goal: p, with hp : p in the context
  exact hp      -- rule Hyp
```

After `intro hp` the goal is the premise of $\to$I. The editor's infoview lists the hypotheses `p : Prop` and `hp : p` on separate lines above the target `⊢ p`: sequent notation, written vertically.

`apply` is $\to$E used backwards: if the goal is `r` and you have `f : q → r`, `apply f` leaves you to prove `q`:

```
/-- `apply f` is rule →E, read bottom-up: if `f : q → r` and the goal is `r`,
it is enough to prove `q`. -/
```

```
theorem imp_trans_tac (p q r : Prop) : (p → q) → (q → r) → (p → r) := by
  intro hpq hqr hp   -- three uses of →I at once
  apply hqr          -- goal was r; now it is q
  apply hpq          -- goal was q; now it is p
  exact hp
```

`constructor` applies the introduction rule of the goal's connective, here ∧I, and leaves one goal per premise; the bullets `·` focus on each goal in turn:

```
/-- `constructor` applies an introduction rule. For `∧` that is ∧I, and it
leaves one goal per premise. The bullets `·` focus on one goal at a time. -/
theorem and_intro_tac (p q : Prop) (hp : p) (hq : q) : p ∧ q := by
  constructor
  · exact hp    -- first premise of ∧I
  · exact hq    -- second premise of ∧I
```

`obtain ⟨hp, hq⟩ := h` eliminates a conjunction: it replaces `h : p ∧ q` with its two parts, which is ∧E1 and ∧E2 at once:

```
/-- `obtain ⟨hp, hq⟩ := h` takes a conjunction apart: both eliminations ∧E₁ and
∧E₂ at once. -/
theorem and_swap_tac (p q : Prop) : p ∧ q → q ∧ p := by
  intro h
  obtain ⟨hp, hq⟩ := h   -- now hp : p and hq : q
  exact ⟨hq, hp⟩         -- ∧I, written as an anonymous constructor
```

`left` and `right` are ∨I1 and ∨I2: they commit to one side of a disjunction:

```
/-- `left` and `right` are the two introduction rules ∨I₁ and ∨I₂: they choose
which side of the disjunction we are going to prove. -/
theorem or_intro_tac (p q : Prop) (hq : q) : p ∨ q := by
  right       -- ∨I₂: the goal becomes q
  exact hq
```

`cases h with` is ∨E. It produces one goal for each way `h` could have been built, each with the corresponding assumption, exactly like the two right-hand premises of the rule:

```
/-- `cases h with` is ∨E: one goal per constructor of `Or`, each with the
assumption that this constructor was used. -/
theorem or_swap_tac (p q : Prop) : p ∨ q → q ∨ p := by
  intro h
  cases h with
  | inl hp =>     -- case 1: h was built by Or.inl, and hp : p
    right
    exact hp
  | inr hq =>     -- case 2: h was built by Or.inr, and hq : q
    left
    exact hq
```

The names `inl` and `inr` are the constructors of `Or` (the functions `Or.inl` and `Or.inr` you saw above). If you know Rust, `cases` will remind you of `match`: one arm per variant, each arm binding the variant's payload. That is not a coincidence, and Chapter 10 explains it.

`rcases` and `rintro` take nested structures apart in one step. The pattern `⟨hp, hq | hr⟩` says "split the conjunction, then split its second component, which is a disjunction, into two cases":

```
/-- `rcases` and `rintro` take apart nested conjunctions and disjunctions in
one step: `⟨_, _⟩` splits an `∧`, and `_ | _` splits an `∨` into two goals. -/
theorem and_or_distrib (p q r : Prop) : p ∧ (q ∨ r) → (p ∧ q) ∨ (p ∧ r) := by
  intro h
```

```
  rcases h with ⟨hp, hq | hr⟩
  · left
    exact ⟨hp, hq⟩
  · right
    exact ⟨hp, hr⟩

/-- `rintro` is `intro` followed by `rcases`. -/
theorem and_or_distrib' (p q r : Prop) : (p ∧ q) ∨ (p ∧ r) → p ∧ (q ∨ r) := by
  rintro (⟨hp, hq⟩ | ⟨hp, hr⟩)
  · exact ⟨hp, Or.inl hq⟩
  · exact ⟨hp, Or.inr hr⟩
```

`exfalso` is ⊥E read upward: whatever the goal is, replace it with `False`, since a proof of `False` proves anything. Here the proof of `False` comes from →E, applying `hnp : ¬p` (which *is* `p → False`) to `hp : p`:

```
/-- `exfalso` is ⊥E read bottom-up: whatever the goal, replace it by `False`.
Then we prove `False` by →E, applying `hnp : p → False` to `hp : p`. -/
theorem not_or_imp (p q : Prop) : ¬p ∨ q → (p → q) := by
  intro h hp
  rcases h with hnp | hq
  · exfalso       -- goal: False
    exact hnp hp  -- ¬p is p → False, so this is →E
  · exact hq
```

`contradiction` is a small automation: it closes any goal when the context is visibly inconsistent, for instance when it contains both `p` and `¬p`. Behind the scenes it uses →E and ⊥E, just as the previous proof did by hand:

```
/-- `contradiction` closes any goal when the context is visibly inconsistent:
it contains `False`, or both `p` and `¬p`. -/
theorem non_contradiction (p q : Prop) (hp : p) (hnp : ¬p) : q := by
  contradiction
```

Negation needs no tactics of its own. To prove `¬p`, `intro` a proof of `p` and derive `False`; to use `¬p`, apply it:

```
/-- Negation is implication into `False`, so its rules are those of `→`:
`intro` proves a negation, and applying it to a proof eliminates it. -/
theorem contrapositive (p q : Prop) : (p → q) → ¬q → ¬p := by
  intro hpq hnq hp   -- the last intro unfolds ¬p into p → False
  exact hnq (hpq hp)

theorem not_and_not (p : Prop) : ¬(p ∧ ¬p) := by
  intro ⟨hp, hnp⟩
  exact hnp hp
```

In `not_and_not`, the pattern after `intro` takes the conjunction apart as it is introduced. `↔` is a structure whose two fields are the two directions, so `constructor` splits it:

```
/-- Currying: a proof of `p ∧ q → r` and a proof of `p → q → r` can be turned
into each other. `↔` is a structure with two fields, built by `constructor`. -/
theorem curry_iff (p q r : Prop) : (p ∧ q → r) ↔ (p → q → r) := by
  constructor
  · intro h hp hq
    exact h ⟨hp, hq⟩
  · intro h ⟨hp, hq⟩
    exact h hp hq
```

Finally, when a step is wrong, Lean says which goal it could not close and why:

```
/--
error: Type mismatch
  hp
has type
  p
but is expected to have type
  q
-/
#guard_msgs in
theorem wrong (p q : Prop) : p → q := by
  intro hp
  exact hp
```

After `intro hp` the goal is `q`, and `hp` has type `p`, so `exact hp` fails: there is no rule that turns a proof of `p` into a proof of an unrelated `q`.

What a tactic proof really is

A tactic script is not the proof; it is a program that *builds* the proof term. The command `#print` shows the term that Lean stored:

```
/--
info: theorem Ch04.and_swap_tac : ∀ (p q : Prop), p ∧ q → q ∧ p :=
fun p q h => And.casesOn h fun hp hq => ⟨hq, hp⟩
-/
#guard_msgs in
#print and_swap_tac

/--
info: theorem Ch04.imp_trans_tac : ∀ (p q r : Prop), (p → q) → (q → r) → p → r :=
fun p q r hpq hqr hp => hqr (hpq hp)
-/
#guard_msgs in
#print imp_trans_tac
```

The second one is literally the term we wrote by hand for `imp_trans`. In the first, `obtain` produced `And.casesOn h fun hp hq => ...`: "take `h` apart into its two components and continue with them". Disjunction is more verbose:

```
/--
info: theorem Ch04.or_swap_tac : ∀ (p q : Prop), p ∨ q → q ∨ p :=
fun p q h => Or.casesOn (motive := fun t => h = t → q ∨ p) h (fun hp h => Or.inr hp) (fun hq
h => Or.inl hq) (Eq.refl h)
-/
#guard_msgs in
#print or_swap_tac
```

The `motive` describes the conclusion of the case analysis; the `h = t →` inside it and the final `Eq.refl h` are bookkeeping that `cases` adds so that each branch can remember which case it is in. What remains is `Or.casesOn h (fun hp => Or.inr hp) (fun hq => Or.inl hq)`: one function per case, which is the ∨E rule. Tactics are a convenient way to write natural-deduction trees, and the kernel checks the trees.

4.4 Harmony

Why these rules and not others? The answer is a design principle that natural deduction makes visible, and it is the conceptual heart of the chapter.

Introductions define, eliminations follow

Gentzen remarked that the introduction rules can be seen as *definitions* of the connectives: $A \wedge B$ is whatever you get from a proof of A and a proof of B, and nothing more. The eliminations must then be *justified* by the introductions: they may extract exactly what an introduction put in. Extracting more proves things that should not be provable; extracting less loses information. The two halves of the requirement are **local soundness** and **local completeness**; together they are **harmony**.

Local soundness: detours can be removed

A **detour** is an introduction rule immediately followed by an elimination of the same connective. Local soundness says that every detour can be removed, leaving a derivation of the same conclusion from the same assumptions. For conjunction, suppose the premises of $\wedge$I have derivations $\mathcal{D}_1$ (of $\Gamma \vdash A$) and $\mathcal{D}_2$ (of $\Gamma \vdash B$). The detour on the left becomes the derivation on the right:

$$\dfrac{\dfrac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\,\wedge\mathsf{I}}{\Gamma \vdash A}\,\wedge\mathsf{E}_1 \qquad \mapsto \qquad \Gamma \vdash A \;\text{ by }\; \mathcal{D}_1$$

We built $A \wedge B$ from a proof of A only to take A out again; we could have used $\mathcal{D}_1$ directly, and $\mathcal{D}_2$ is thrown away. $\wedge$E2 is symmetric. For implication, let $\mathcal{D}_1$ derive $\Gamma, A \vdash B$ and $\mathcal{D}_2$ derive $\Gamma \vdash A$:

$$\dfrac{\dfrac{\Gamma, A \vdash B}{\Gamma \vdash A \to B}\,\to\mathsf{I} \qquad \Gamma \vdash A}{\Gamma \vdash B}\,\to\mathsf{E} \qquad \mapsto \qquad \Gamma \vdash B \;\text{ by }\; \mathcal{D}_1[\mathcal{D}_2/A]$$

We proved B from the assumption A, packaged that as $A \to B$, and applied it to a proof of A. The direct proof, written $\mathcal{D}_1[\mathcal{D}_2/A]$, is $\mathcal{D}_1$ with every use of the assumption A (every Hyp leaf for it) replaced by a copy of $\mathcal{D}_2$. This is the most important reduction in the book. Proving a lemma and immediately applying it is a function definition followed by a call, and removing the detour is substituting the argument for the parameter. In the λ-calculus of Chapter 7 it is written

$$(\lambda x.\, e)\; v \to e[v/x]$$

and called **β-reduction**. The basic step of computation is local soundness for implication: computation was not added to logic from outside, it is the act of removing detours from proofs.

For disjunction, if the disjunction was built by $\vee$I1 from a derivation $\mathcal{D}$ of $\Gamma \vdash A$, we know which case of $\vee$E applies, and can take it. With $\mathcal{E}_1$ deriving the first case, $\Gamma, A \vdash C$:

$$\dfrac{\dfrac{\Gamma \vdash A}{\Gamma \vdash A \vee B}\,\vee\mathsf{I}_1 \qquad \Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma \vdash C}\,\vee\mathsf{E} \qquad \mapsto \qquad \Gamma \vdash C \;\text{ by }\; \mathcal{E}_1[\mathcal{D}/A]$$

In a programming language this is a `match` on a value whose variant is known: `match Left(v) { Left(x) => e1, Right(y) => e2 }` evaluates to `e1` with `v` for `x`. Falsity has no introduction rule, so there is no detour to remove and local soundness holds vacuously; the same goes for truth, which has no elimination.

Local completeness: nothing is lost

Local completeness asks the opposite question: are the eliminations strong enough? Given *any* derivation of a formula, can we take it apart with the eliminations and rebuild it with the introductions? For conjunction, given any derivation $\mathcal{D}$ of $\Gamma \vdash A \wedge B$:

$$\Gamma \vdash A \wedge B \quad \mapsto \quad \dfrac{\dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\,\wedge \mathsf{E}_1 \qquad \dfrac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\,\wedge \mathsf{E}_2}{\Gamma \vdash A \wedge B}\,\wedge \mathsf{I}$$

where both copies of $\Gamma \vdash A \wedge B$ on the right are proved by $\mathcal{D}$.

For implication, from $\mathcal{D}$ proving $A \to B$ we assume A, apply $\mathcal{D}$ with $\to$E and discharge with $\to$I. For disjunction, we split $\mathcal{D}$ with $\vee$E and rebuild each case with the matching $\vee$I. For truth, any derivation of $\top$ can be replaced by $\top$I. On programs these expansions are called **η**: a pair `p` is the same as `(p.0, p.1)`, a function `f` the same as `|x| f(x)`, and any value of type `()` is `()`.

Condition	Proof-theoretic content	Computational content
local soundness	detours can be removed	β-reduction
local completeness	eliminations recover everything	η-expansion

The two halves of harmony are exactly the two halves of the equational theory of the λ-calculus. You will meet β and η again in Chapters 7, 8 and 19.

Harmony in Lean

Lean takes Gentzen's remark literally. When you define a proposition with `inductive`, you give only its introduction rules, the constructors, and Lean *derives* the elimination rule. The file `code/lean/Book/Ch04/Harmony.lean` defines our own copies of the connectives:

```
/-- Conjunction: one introduction rule with two premises. -/
inductive MyAnd (a b : Prop) : Prop where
  | intro : a → b → MyAnd a b

/-- Disjunction: two introduction rules. -/
inductive MyOr (a b : Prop) : Prop where
  | inl : a → MyOr a b
  | inr : b → MyOr a b

/-- Truth: one introduction rule with no premises. -/
inductive MyTrue : Prop where
  | intro : MyTrue

/-- Falsity: no introduction rules at all. -/
inductive MyFalse : Prop
```

For each one Lean generates a *recursor*, its most general elimination rule:

```
/--
info: @MyOr.rec : ∀ {a b : Prop} {motive : MyOr a b → Prop},
  (∀ (a_1 : a), motive …) → (∀ (a_1 : b), motive …) → ∀ (t : MyOr a b), motive t
-/
#guard_msgs in
#check @MyOr.rec

/--
info: @MyTrue.rec : {motive : MyTrue → Sort u_1} → motive MyTrue.intro → (t : MyTrue) →
motive t
```

```
-/
#guard_msgs in
#check @MyTrue.rec

/-- info: MyFalse.rec : (motive : MyFalse → Sort u_1) → (t : MyFalse) → motive t -/
#guard_msgs in
#check @MyFalse.rec
```

The recursor of `MyOr` takes one function per constructor and proves `motive t` for any `t : MyOr a b`. The motive is the conclusion C of $\vee$E, here allowed to depend on the proof being eliminated (the `…` are proofs that Lean does not print). The recursor of `MyTrue` needs the motive in the single case `MyTrue.intro` and gives back nothing new: the useless elimination that $\top$ does without. The recursor of `MyFalse` has *no* cases and proves any motive: it is $\bot$E. With a constant motive, the familiar rules fall out:

```
/-- ∧E₁, derived from the recursor: take the pair apart, keep the left part. -/
theorem MyAnd.left {a b : Prop} (h : MyAnd a b) : a :=
  MyAnd.rec (motive := fun _ => a) (fun ha _ => ha) h

/-- ∨E, derived from the recursor with a constant motive `c`. -/
theorem MyOr.elim {a b c : Prop} (h : MyOr a b) (left : a → c) (right : b → c) : c :=
  MyOr.rec (motive := fun _ => c) left right h

/-- ⊥E, derived from the recursor: zero cases to handle. -/
theorem MyFalse.elim {c : Prop} (h : MyFalse) : c :=
  MyFalse.rec (motive := fun _ => c) h
```

Because elimination is derived from introduction, every inductive proposition in Lean is harmonious by construction. You cannot define `tonk` (next section) with `inductive`: you can only state its introduction rule, and Lean will hand you the matching, harmless elimination.

Local soundness and completeness are invisible on proofs in Lean, because two proofs of the same proposition are always considered equal (proof irrelevance, Chapter 10). They are visible on *data*, where Lean's evaluator performs exactly the reductions above:

```
/-! Local soundness, on data: an introduction followed by an elimination
computes away. All three hold by `rfl`: Lean evaluates both sides. -/
example (α β : Type) (a : α) (b : β) : (a, b).1 = a := rfl

example (α β : Type) (f : α → β) (a : α) : (fun x => f x) a = f a := rfl

example (α β γ : Type) (f : α → γ) (g : β → γ) (a : α) :
    Sum.elim f g (Sum.inl a) = f a := rfl
```

```
/-! Local completeness, on data: eliminating and re-introducing gives back
what we started with. For pairs, functions and `Unit` Lean knows this by
`rfl` (definitional η); for sums it needs a case analysis. -/
example (α β : Type) (p : α × β) : (p.1, p.2) = p := rfl

example (α β : Type) (f : α → β) : (fun x => f x) = f := rfl

example (u : Unit) : u = () := rfl

example (α β : Type) (s : α ⊕ β) : Sum.elim Sum.inl Sum.inr s = s := by
  cases s <;> rfl
```

All the β-rules hold by `rfl`, that is, by computation. So do the η-rules for pairs, functions and `Unit`, which Lean builds into its notion of definitional equality. The η-rule for sums does not: it needs `cases s`

and then `rfl` in each case (the combinator `<;>` runs the tactic after it on every goal produced by the one before it). The odd one out is again disjunction, whose elimination has a different shape, as we will see after the next section.

Tonk

In 1960 Arthur Prior challenged the idea that rules can define the meaning of a connective. If they can, he said, consider the connective `tonk`, with an introduction rule borrowed from $\vee$ and an elimination rule borrowed from $\wedge$:

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \mathit{tonk} B}\ \text{tonk-I} \qquad\qquad \frac{\Gamma \vdash A \mathit{tonk} B}{\Gamma \vdash B}\ \text{tonk-E}$$

Each rule looks innocent alone. Together they prove everything:

$$\dfrac{\dfrac{\dfrac{}{\vdash \top}\ \top\text{I}}{\vdash \top \mathit{tonk} \bot}\ \text{tonk-I}}{\vdash \bot}\ \text{tonk-E}$$

and from $\bot$, by $\bot$E, any formula at all. A logic in which everything is provable is useless. Harmony is the precise diagnosis: the detour tonk-I followed by tonk-E cannot be removed, because we have a derivation of A, need one of B, and nothing connects them. Prior's challenge was that "meaning given by rules" licenses nonsense; the answer is that only harmonious rules give meaning, and harmony is a checkable property.

In Lean we can state the catastrophe as a theorem: no operation on propositions satisfies both of tonk's rules, because any such operation would prove `False`. And no truth table can make both rules sound:

```
/-- No operation on propositions obeys both of Prior's rules for `tonk`:
if one did, `False` would be provable. -/
theorem no_tonk (tonk : Prop → Prop → Prop)
    (tonkI : ∀ {a b : Prop}, a → tonk a b)
    (tonkE : ∀ {a b : Prop}, tonk a b → b) : False :=
  tonkE (tonkI trivial)

/-- No truth table for `tonk` makes both rules sound: tonk-I needs
`a → tonk a b`, tonk-E needs `tonk a b → b`, and the row `a = true`,
`b = false` cannot satisfy both. -/
theorem no_tonk_table : ¬ ∃ f : Bool → Bool → Bool,
    (∀ a b, a = true → f a b = true) ∧ (∀ a b, f a b = true → b = true) := by
  intro ⟨f, hI, hE⟩
  have h : false = true := hE true false (hI true false rfl)
  contradiction
```

The proof of `no_tonk` is the derivation above, written as a term. In `no_tonk_table`, tonk-I demands that `f true b` be `true` and tonk-E demands that `f a false` be `false`, and the row `a = true, b = false` cannot satisfy both: `contradiction` closes the goal because the hypothesis `false = true` equates two different constructors of `Bool`.

Why $\vee$E and $\bot$E look strange

Most people find $\vee$E odd on first contact: $\wedge$E simply extracts a component, while $\vee$E has three premises and an arbitrary conclusion C. Harmony forces this shape. $A \vee B$ may have been introduced by either of two rules, and the elimination cannot know which. The strongest thing it can do is require the user

to handle *both* cases, and let them produce whatever they like. That is what makes local soundness work for both introductions at once. The pattern is general:

Introduction rules	Shape of the elimination	Examples
one	projections: take out what was put in	$\wedge$, $\to$
several	case analysis: handle every introduction	$\vee$
none	case analysis with zero cases: conclude anything	$\bot$
one, with no premises	nothing to take out: no elimination	$\top$

$\bot$E is therefore not a special license for absurdity. It is the case analysis of $\vee$E with zero cases, and Lean's `MyFalse.rec` above has exactly that shape. Rust has the same thing: an `enum` with no variants can be matched with no arms, and the match has any type you like.

```
enum Void {}

fn absurd<T>(v: Void) -> T {
    match v {} // zero variants, zero arms: this is rule ⊥E
}
```

(This fragment is compiled as part of `code/rust/examples/ch05_negation.rs`, which Chapter 5 discusses.) Is $\bot$E dangerous? Only in a context that is already contradictory: the consistency theorem below shows that $\bot$ is never provable from no assumptions. Explosion only says that an impossible situation constrains nothing, just as an unreachable `match` arm can claim any type because it never runs.

4.5 Normalization, subformulas and consistency

A derivation with no detours is called **normal**. Removing one detour can create others (substituting a derivation into a hole can place an introduction right above an elimination), so it is not obvious that repeatedly removing detours ever stops. It does:

- **Normalization theorem** (Dag Prawitz). Every derivation can be reduced to a normal derivation of the same sequent. Weak normalization says that some sequence of reductions terminates; strong normalization says that every sequence does, whatever detour you choose to remove first. Both hold for the system of this chapter.

Normal derivations have a remarkable property:

- **Subformula property.** Every formula that occurs in a normal derivation is a subformula of the conclusion or of an open assumption.

A normal proof never invents anything: it is built from pieces of its conclusion and assumptions. (The full theorem needs a few extra reductions, called *permutative conversions*, that move $\vee$E and $\bot$E out of the way; we will not need them.) Detours are where "foreign" formulas come from, as the Rust normalizer will show.

From these two theorems follows something that sounds like it should need a semantic argument:

- **Consistency.** There is no derivation of $\vdash \bot$.

Proof sketch. Suppose there were one; by normalization there is a normal one. Its last rule cannot be an introduction, because $\bot$ has none, nor Hyp, because the context is empty. So it is an elimination. Follow its *major* premise (the one containing the eliminated connective) upward, then that rule's major premise, and so on. In a normal derivation no major premise is the conclusion of an introduction, so the path runs through eliminations only, and along it the context never changes (eliminations extend the context only in the case premises of $\vee$E). The path ends at a Hyp, which needs a formula in the context, but the context is empty.

Consistency, normally a question about meaning, follows from a syntactic fact about how proofs simplify. Each of these results has a programming reading, which Part II develops:

Result	Statement	Reading for programs
weak normalization	some reduction sequence ends	some evaluation order terminates
strong normalization	every reduction sequence ends	every evaluation order terminates
subformula property	normal proofs use only subformulas	values are built from their types' parts
consistency	$\bot$ has no closed proof	the empty type has no closed value

Rust, with `loop {}`, has no normalization theorem, and indeed `loop {}` has every type including the empty one. Chapter 10 explains why that makes Rust useless as a logic and harmless as a programming language.

4.6 Sequent calculus and cut elimination

Gentzen gave a second presentation of the same logic, the **sequent calculus** (LJ, for the intuitionistic version). It keeps the introductions, now called *right rules*, and replaces each elimination by a *left rule* that introduces the connective on the left of $\vdash$, among the assumptions. For conjunction and implication:

$$\frac{\Gamma, A, B \vdash C}{\Gamma, A \wedge B \vdash C}\ \wedge\mathsf{L} \qquad\qquad \frac{\Gamma \vdash A \qquad \Gamma, B \vdash C}{\Gamma, A \to B \vdash C}\ \to\mathsf{L}$$

Read $\wedge$L as "an assumption $A \wedge B$ can be split into two assumptions", and $\to$L as "to use an assumption $A \to B$, prove A, then continue with B as an assumption". Every rule of the sequent calculus builds up formulas, never takes them apart, with one exception:

$$\frac{\Gamma \vdash A \qquad \Gamma, A \vdash C}{\Gamma \vdash C}\ \mathsf{Cut}$$

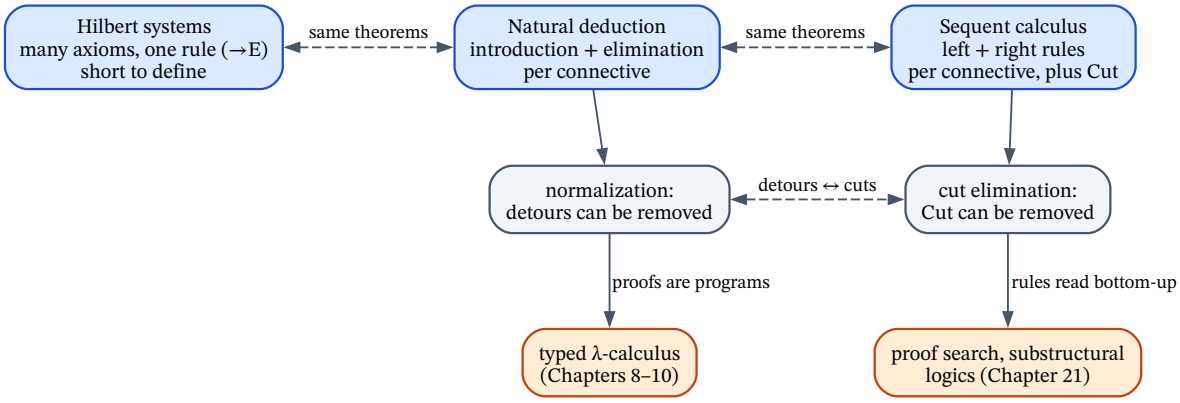
Cut is "prove a lemma A, then use it". Gentzen's main theorem, the *Hauptsatz*, is that cut is unnecessary:

- **Cut elimination.** Every derivation in the sequent calculus can be transformed into one that does not use Cut.

Cut elimination and normalization are two faces of one phenomenon: translating between the two systems maps cuts to detours. The price is size. Cut-free proofs can be enormously larger than proofs

with cuts, which is the proof-theoretic reason why mathematicians use lemmas even though, in principle, they need not.

Why have two systems? Because they are good at different things.



Three presentations of the same logic, the theorem that makes each one tick, and where each one leads in this book.

Natural deduction is where proofs look like programs. The sequent calculus is where proof *search* lives: in a cut-free proof every rule, read bottom-up, only takes apart the goal and the assumptions, so a program can search by trying the finitely many rules that apply. It is also where **substructural logic** was born: the phrase means "a sequent calculus with some structural rules removed", and Chapters 6 and 21 use exactly this vocabulary. *Hilbert systems*, with many axioms and only modus ponens, are short to define and awkward to use; we will not need them.

4.7 A proof checker in Rust

It is instructive to build a proof checker of our own, in the language you know, and watch it reject bad proofs. The crate `natded` (in `code/rust/crates/natded`) has a formula type with a parser and printer, proof trees, the checker with its error messages, truth tables, and detour removal.

Formulas and proofs as data

Formulas are an `enum`, one variant per case of the grammar, plus Prior's `tonk`, which the checker will reject unless we ask for it:

```
/// A formula of propositional logic.
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub enum Formula {
    /// An atomic proposition such as `A` or `Raining`.
    Atom(String),
    /// Truth, `⊤`.
    Top,
    /// Falsity, `⊥`.
    Bot,
    /// Conjunction, `A ∧ B`.
    And(Box<Formula>, Box<Formula>),
    /// Disjunction, `A ∨ B`.
    Or(Box<Formula>, Box<Formula>),
    /// Implication, `A → B`. Negation `¬A` is `A → ⊥`.
```

```
    Imp(Box<Formula>, Box<Formula>),
    /// Prior's `A tonk B`: not a real connective, see Chapter 4.
    Tonk(Box<Formula>, Box<Formula>),
}
```

A proof is a tree with one variant per rule. It is written in Gentzen's style: assumptions have names, and the rules that discharge an assumption (→I, and each case of ∨E) choose the name:

```
/// A proof tree. Each variant is one rule of natural deduction.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Proof {
    /// Hyp: use the assumption with this name.
    Hyp(String),
    /// ⊤I: truth, from no premises.
    TopI,
    /// ⊥E: from a proof of `⊥`, the given formula.
    BotE(Box<Proof>, Formula),
    /// ∧I: from proofs of `A` and of `B`, `A ∧ B`.
    AndI(Box<Proof>, Box<Proof>),
    /// ∧E1: from `A ∧ B`, `A`.
    AndE1(Box<Proof>),
    /// ∧E2: from `A ∧ B`, `B`.
    AndE2(Box<Proof>),
    /// ∨I1: from `A`, `A ∨ B`; the formula is `B`.
    OrI1(Box<Proof>, Formula),
    /// ∨I2: from `B`, `A ∨ B`; the formula is `A`.
    OrI2(Formula, Box<Proof>),
    /// ∨E: from `A ∨ B`, a proof of `C` assuming `A` (named `left.0`) and a
    /// proof of `C` assuming `B` (named `right.0`), conclude `C`.
    OrE {
        scrutinee: Box<Proof>,
        left: (String, Box<Proof>),
        right: (String, Box<Proof>),
    },
    /// →I: assume `A` under the given name, prove `B`, conclude `A → B`.
    /// The assumption is discharged: it cannot be used outside `body`.
    ImpI {
        name: String,
        assumption: Formula,
        body: Box<Proof>,
    },
    /// →E (modus ponens): from `A → B` and `A`, `B`.
    ImpE(Box<Proof>, Box<Proof>),
    /// tonk-I: from `A`, `A tonk B`; the formula is `B`.
    TonkI(Box<Proof>, Formula),
    /// tonk-E: from `A tonk B`, `B`.
    TonkE(Box<Proof>),
}
```

A few rules carry a formula. →I records the formula of the assumption it introduces (Gentzen's $[A]^u$), ∨I1 records the other disjunct, and ⊥E records its conclusion: these are formulas that cannot be computed from the premises. Everything else the checker works out. Short constructor functions (`hyp`, `and_i`, `imp_i`, ...) make proofs readable. Here is the derivation of $\vdash A \wedge B \to B \wedge A$, from the example program `code/rust/crates/natded/examples/ch04_proof_checker.rs`:

```
// ⊢ A ∧ B → B ∧ A. Assume A ∧ B under the name `u`, take it apart twice
// (∧E2 and ∧E1), put the parts back together in the other order (∧I),
// and discharge `u` (→I).
let and_comm = imp_i("u", f("A ∧ B"), and_i(and_e2(hyp("u")), and_e1(hyp("u"))));
let d1 = checker
    .prove(&[], &f("A ∧ B → B ∧ A"), &and_comm)
    .expect("a correct proof");
print!("{d1}");
```

The checker

`Checker::check` walks the tree, computing the formula each node proves and checking that each rule is applied to premises of the right shape. It returns a `Derivation`, the same tree with the full sequent at every node, or a `CheckError`. Here are the two rules for implication:

```
Proof::ImpI {
    name,
    assumption,
    body,
} => {
    // The body may use the new assumption; the conclusion may not.
    let inner = extended(context, name, assumption);
    let d = self.premise(&inner, body, path, 1)?;
    let c = Formula::imp(assumption.clone(), d.conclusion.clone());
    labels.push(name.clone());
    premises.push(d);
    c
}
Proof::ImpE(p, q) => {
    let d1 = self.premise(context, p, path, 1)?;
    let Formula::Imp(a, b) = &d1.conclusion else {
        let found = d1.conclusion;
        let kind = shape("first premise", "an implication", found);
        return Err(fail(path, kind));
    };
    let d2 = self.premise(context, q, path, 2)?;
    if d2.conclusion != **a {
        let kind = ErrorKind::ArgumentMismatch {
            implication: d1.conclusion.clone(),
            found: d2.conclusion,
        };
        return Err(fail(path, kind));
    }
    let c = (**b).clone();
    premises.extend([d1, d2]);
    c
}
```

For →I, the checker extends the context with the named assumption, checks the body there, and concludes $A \to B$ in the *original* context: discharge is the scoping of a variable. For →E, it insists that the first premise proves an implication and the second its antecedent. The other rules follow the same pattern. Printed, the derivation of $\vdash A \wedge B \to B \wedge A$ is the earlier tree in sequent notation, with each rule (and the assumption it uses or discharges) on the right:

```
⊢ A ∧ B → B ∧ A          →I u
└─ A ∧ B ⊢ B ∧ A         ∧I
   ├─ A ∧ B ⊢ B          ∧E2
   │  └─ A ∧ B ⊢ A ∧ B   Hyp u
   └─ A ∧ B ⊢ A          ∧E1
      └─ A ∧ B ⊢ A ∧ B   Hyp u
```

Each sequent is proved by the rule on its line from the sequents indented just below it. The context on the second line is $A \wedge B$ because →I added it; the root's context is empty because →I discharged it.

Error messages

A checker is only as useful as its error messages. Three wrong proofs:

```
// Three mistakes. First: projecting from something that is not a pair.
let wrong_shape = and_e1(hyp("u"));
let e1 = checker
```

```
    .check(&[("u", f("A → B"))], &wrong_shape)
    .unwrap_err();
println!("{e1}");

// Second: using an assumption after the →I that discharged it.
let out_of_scope = and_i(imp_i("u", f("A"), hyp("u")), hyp("u"));
let e2 = checker.check(&[], &out_of_scope).unwrap_err();
println!("{e2}");

// Third: modus ponens with the wrong argument.
let wrong_argument = imp_i("b", f("B"), imp_e(hyp("g"), hyp("b")));
let e3 = checker
    .check(&[("g", f("A → C"))], &wrong_argument)
    .unwrap_err();
println!("{e3}");
```

```
error in ∧E1 at root:
  the premise must prove a conjunction, but it proves `A → B`
error in Hyp at root.2:
  no assumption named `u` is available here (none are)
error in →E at root.1:
  the implication is `A → C`, so the argument must prove `A`, but it proves `B`
```

Each error names the failing rule and its position (`root.2.1` would mean "the first premise of the second premise of the root"). The first proof applies ∧E1 to an implication. The second uses `u` in the second premise of ∧I, but `u` was discharged by the ⟶I in the first premise: an assumption exists only inside the subproof that discharges it, as a variable exists only in its scope. The third applies $A \to C$ to a proof of B. These messages read like Rust's E0308 "mismatched types", and Chapter 10 explains why: a type checker is a proof checker.

Truth tables and soundness, tested

The crate also evaluates formulas in the classical two-valued way, the truth tables you may know from digital logic: an atom is true or false, and each connective computes a truth value from those of its parts.

```
/// The truth value of `f` in the row `v`. Atoms missing from `v` are false.
/// Returns `None` if `f` mentions `tonk`, which has no truth table.
pub fn eval(f: &Formula, v: &Valuation) -> Option<bool> {
    if f.mentions_tonk() {
        return None;
    }
    Some(eval_with(f, v, &[[false; 2]; 2]))
}

/// The truth value of `f`, reading `tonk` from the given table.
pub fn eval_with(f: &Formula, v: &Valuation, tonk: &TonkTable) -> bool {
    match f {
        Formula::Atom(name) => v.get(name).copied().unwrap_or(false),
        Formula::Top => true,
        Formula::Bot => false,
        Formula::And(a, b) => eval_with(a, v, tonk) && eval_with(b, v, tonk),
        Formula::Or(a, b) => eval_with(a, v, tonk) || eval_with(b, v, tonk),
        Formula::Imp(a, b) => !eval_with(a, v, tonk) || eval_with(b, v, tonk),
        Formula::Tonk(a, b) => {
            tonk[eval_with(a, v, tonk) as usize][eval_with(b, v, tonk) as usize]
        }
    }
}
```

(`eval_with` also takes a candidate truth table for `tonk`; we will need it at the end of this section.) Truth tables give a *semantics*, an account of what formulas mean that does not mention proofs. A proof system is **sound** for a semantics when everything it proves is true in that semantics. For truth tables this means: if $\Gamma \vdash A$ is derivable, then every row that makes all of Γ true makes A true. The example program checks this on the proofs it builds, and uses the contrapositive to show that a formula is *not* provable:

```
// Soundness, tested: every sequent the checker accepts is true in every
// row of its truth table.
for d in [&d1, &d2] {
    let hyps: Vec<Formula> = d.context.iter().map(|(_, a)| a.clone()).collect();
    assert!(counterexample(&hyps, &d.conclusion).is_none());
}
print!("{}", truth_table(&f("A ∧ B → B ∧ A")));

// A formula with a counterexample cannot have a proof: by soundness, the
// proof would make it true in the falsifying row.
let not_valid = f("A ∨ B → A");
let row = counterexample(&[], &not_valid).expect("not a tautology");
println!("{not_valid} is false when {}", show_valuation(&row));
```

```
A B │ A ∧ B → B ∧ A
F F │ T
F T │ T
T F │ T
T T │ T
A ∨ B → A is false when A = F, B = T
```

The second part is the useful one: since $A \vee B \to A$ is false in the row $A = F$, $B = T$, soundness guarantees that no proof exists. A Rust program can only check soundness proof by proof; in Lean we will prove it once and for all.

Normalizing proofs

The module `natded::normalize` implements the local-soundness reductions. Its core is a function that recognizes a detour at the root of a proof and removes it:

```
/// If `p` is itself a detour, removes it and names the reduction.
fn contract(p: &Proof) -> Option<(Proof, &'static str)> {
    match p {
        Proof::AndE1(q) => match &**q {
            // ∧I followed by ∧E1: keep the first component, drop the second.
            Proof::AndI(a, _) => Some(((**a).clone(), "∧I/∧E1")),
            _ => None,
        },
        Proof::AndE2(q) => match &**q {
            Proof::AndI(_, b) => Some(((**b).clone(), "∧I/∧E2")),
            _ => None,
        },
        Proof::ImpE(f, arg) => match &**f {
            // →I followed by →E: plug the argument into the body.
            Proof::ImpI { name, body, .. } => {
                Some((substitute(body, name, arg), "→I/→E"))
            }
            _ => None,
        },
        Proof::OrE {
            scrutinee,
            left,
            right,
        } => match &**scrutinee {
```

```
            // vI followed by vE: we know which case happened; take it.
            Proof::OrI1(a, _) => Some((substitute(&left.1, &left.0, a), "vI1/vE")),
            Proof::OrI2(_, b) => Some((substitute(&right.1, &right.0, b), "vI2/vE")),
            _ => None,
        },
        // tonk-I followed by tonk-E is a detour, but there is nothing to
        // reduce it to: the premise proves A and we need B.
        _ => None,
    }
}
```

These are the reductions of the harmony section, line by line. →I/→E calls `substitute`, which replaces every use of the assumption by the argument proof, renaming inner assumptions where needed so that the argument's own open assumptions are not captured (substitution in the λ-calculus needs the same care, Chapter 7). `normalize` applies `contract` anywhere in the tree until no detour is left. The example `ch04_harmony.rs` proves $A \wedge B \vdash B \wedge A$ through a lemma:

```
// A ∧ B ⊢ B ∧ A, proved by first proving a general lemma A ∧ B → B ∧ A
// (→I) and then applying it to the assumption `u` (→E): a detour.
let lemma = imp_i("x", f("A ∧ B"), and_i(and_e2(hyp("x")), and_e1(hyp("x"))));
let with_detour = imp_e(lemma, hyp("u"));
let hyps = [("u", f("A ∧ B"))];
let before = checker.check(&hyps, &with_detour).expect("correct");
print!("{before}");
println!("not subformulas: {}", show(&foreign_formulas(&before)));
```

```
A ∧ B ⊢ B ∧ A                    →E
├─ A ∧ B ⊢ A ∧ B → B ∧ A         →I x
│  └─ A ∧ B, A ∧ B ⊢ B ∧ A       ∧I
│     ├─ A ∧ B, A ∧ B ⊢ B        ∧E2
│     │  └─ A ∧ B, A ∧ B ⊢ A ∧ B  Hyp x
│     └─ A ∧ B, A ∧ B ⊢ A        ∧E1
│        └─ A ∧ B, A ∧ B ⊢ A ∧ B  Hyp x
└─ A ∧ B ⊢ A ∧ B                  Hyp u
not subformulas: {A ∧ B → B ∧ A}
```

The lemma $A \wedge B \to B \wedge A$ does not occur in the sequent, and `foreign_formulas` reports it: the subformula property fails. (Notice also that the context of the lemma's body contains $A \wedge B$ twice, once as `u` and once as `x`: contexts are lists.) One reduction later:

```
let result = normalize(&with_detour, 100);
println!("reductions: {:?}", result.steps);
let after = checker.check(&hyps, &result.proof).expect("still correct");
print!("{after}");
println!("not subformulas: {}", show(&foreign_formulas(&after)));
assert_eq!(before.sequent(), after.sequent());
```

```
reductions: ["→I/→E"]
A ∧ B ⊢ B ∧ A         ∧I
├─ A ∧ B ⊢ B          ∧E2
│  └─ A ∧ B ⊢ A ∧ B   Hyp u
└─ A ∧ B ⊢ A          ∧E1
   └─ A ∧ B ⊢ A ∧ B   Hyp u
not subformulas: {}
```

The normal proof is the direct one, it proves the same sequent, and it uses only subformulas. The checker re-checks it: reduction preserves correctness, which is the logical form of a property you will meet as *preservation* in Chapter 9.

Adding tonk

Now the experiment: `Checker::with_tonk()` also accepts Prior's two rules.

```
// ⊤ ⊢ ⊤ tonk ⊥ by tonk-I, then ⊥ by tonk-E.
let absurd = tonk_e(tonk_i(top_i(), Formula::Bot));
match checker.check(&[], &absurd) {
    Ok(_) => unreachable!("intuitionistic logic has no tonk"),
    Err(e) => println!("{e}"),
}
let tonk_checker = Checker::with_tonk();
let d = tonk_checker
    .check(&[], &absurd)
    .expect("accepted with tonk");
print!("{d}");

// Now every formula has a proof: wrap the proof of ⊥ in ⊥E.
let anything = bot_e(absurd.clone(), f("A ∧ ¬A"));
let d = tonk_checker
    .check(&[], &anything)
    .expect("accepted with tonk");
println!("accepted: {}", d.sequent());
```

```
error in tonk-E at root:
  this rule is not part of intuitionistic natural deduction
⊢ ⊥             tonk-E
└─ ⊢ ⊤ tonk ⊥   tonk-I
   └─ ⊢ ⊤       ⊤I
accepted: ⊢ A ∧ ¬A
```

The ordinary checker stops at the first rule it does not know, tonk-E at the root. The extended checker accepts a proof of $\vdash \bot$, and with one more $\bot$E a proof of $A \wedge \neg A$. Two diagnoses, one semantic and one proof-theoretic:

```
// Soundness is gone: ⊥ is false in every row of every truth table...
assert!(counterexample(&[], &Formula::Bot).is_some());
// ...and no truth table for tonk could rescue it.
let tables = all_tables();
let intro = tables.iter().filter(|t| tonk_intro_sound(t)).count();
let elim = tables.iter().filter(|t| tonk_elim_sound(t)).count();
let both = tables
    .iter()
    .filter(|t| tonk_intro_sound(t) && tonk_elim_sound(t))
    .count();
println!(
    "truth tables for tonk: {intro} fit tonk-I, {elim} fit tonk-E, {both} fit both"
);

// Local soundness fails: the detour tonk-I/tonk-E cannot be reduced.
let stuck = normalize(&absurd, 100);
println!("reductions: {:?}, stuck: {:?}", stuck.steps, stuck.stuck);
```

```
truth tables for tonk: 4 fit tonk-I, 4 fit tonk-E, 0 fit both
reductions: [], stuck: ["tonk-I/tonk-E"]
```

Of the 16 possible truth tables for a binary connective, four make tonk-I sound (those with $A \mathit{tonk} B$ true whenever A is, such as disjunction) and four make tonk-E sound (those with it false whenever B is, such as conjunction), but none does both: this is the Rust twin of Lean's `no_tonk_table`. And the normalizer finds the detour but has no reduction for it. The checker itself has no bug; it faithfully implements a broken logic.

4.8 A deep embedding in Lean

The Rust checker tests soundness on examples. To prove it for every derivation, we need derivations to be mathematical objects. So far we have used Lean's own `Prop` and `∧`: a **shallow embedding**, in which the logic we study is Lean's logic. That is convenient for proving things *in* the logic, but not *about* it: there is no Lean statement "this proposition has no natural-deduction proof", because Lean's propositions are not data we can inspect. A **deep embedding** defines the syntax and the rules as ordinary inductive types, as Chapter 3 did for `Arith`; then formulas and derivations are values, and we can prove theorems about all of them. The code is in `code/lean/Book/Ch04/Deep.lean`.

Formulas and derivability

Formulas are an inductive type, with atoms numbered rather than named, and negation defined:

```
/-- Propositional formulas. Atoms are numbered: `var 0`, `var 1`, ... -/
inductive Formula where
  | var : Nat → Formula
  | top : Formula
  | bot : Formula
  | and : Formula → Formula → Formula
  | or  : Formula → Formula → Formula
  | imp : Formula → Formula → Formula
  deriving DecidableEq, Repr

/-- Negation is not a constructor: `¬A` abbreviates `A → ⊥`. -/
def Formula.neg (A : Formula) : Formula := .imp A .bot
```

```
/-- Three atoms to write examples with. -/
def P : Formula := .var 0
def Q : Formula := .var 1
def R : Formula := .var 2
```

The judgment $\Gamma \vdash A$ becomes an inductive predicate with one constructor per rule, the same technique as the typing judgment `HasType` of Chapter 3:

```
/-- `Proves Γ A` is the hypothetical judgment `Γ ⊢ A` of intuitionistic natural
deduction, with one constructor per rule. The context is a list, and the newest
assumption is at its head: the `A :: Γ` of the code is the `Γ, A` of the text. -/
inductive Proves : List Formula → Formula → Prop where
  | hyp   {Γ A}     : A ∈ Γ → Proves Γ A
  | topI  {Γ}       : Proves Γ .top
  | botE  {Γ C}     : Proves Γ .bot → Proves Γ C
  | andI  {Γ A B}   : Proves Γ A → Proves Γ B → Proves Γ (.and A B)
  | andE1 {Γ A B}   : Proves Γ (.and A B) → Proves Γ A
  | andE2 {Γ A B}   : Proves Γ (.and A B) → Proves Γ B
  | orI1  {Γ A B}   : Proves Γ A → Proves Γ (.or A B)
  | orI2  {Γ A B}   : Proves Γ B → Proves Γ (.or A B)
  | orE   {Γ A B C} : Proves Γ (.or A B) → Proves (A :: Γ) C → Proves (B :: Γ) C →
                      Proves Γ C
  | impI  {Γ A B}   : Proves (A :: Γ) B → Proves Γ (.imp A B)
  | impE  {Γ A B}   : Proves Γ (.imp A B) → Proves Γ A → Proves Γ B
```

Compare each constructor with its rule: the arguments are the premises, the result type is the conclusion, and the implicit arguments in braces are the metavariables. The context is a list, and an assumption added by $\to$I or $\vee$E goes at the front: `A :: Γ` in the code is Γ, A on paper. A value of type `Proves Γ A` *is* a derivation tree.

Derivations as terms and as tactic scripts

Here is $A \wedge B \vdash B \wedge A$ once more, now as a value of the deep embedding:

```
/-- The derivation of `P ∧ Q ⊢ Q ∧ P`, written as a term. Each `.hyp` needs a
proof that the formula is in the context; `List.Mem.head _` says "it is the
first element". -/
theorem and_comm_derivation : Proves [.and P Q] (.and Q P) :=
  .andI (.andE2 (.hyp (List.Mem.head _))) (.andE1 (.hyp (List.Mem.head _)))
```

The term has the shape of the tree: `.andI` at the root, `.andE2` and `.andE1` below it, `.hyp` at the leaves. Each `.hyp` needs a proof that the formula is in the context; `List.Mem.head _` says "it is the first element of the list". The same derivation can be built bottom-up with tactics, applying one constructor at a time:

```
/-- The same derivation under no assumptions, built bottom-up with `apply`.
Elimination rules have a formula in their premise that does not occur in their
conclusion (the `A` of ∧E₂ below), so we must supply it with `(A := P)`. -/
theorem and_comm_closed : Proves [] (.imp (.and P Q) (.and Q P)) := by
  apply Proves.impI           -- context is now [P ∧ Q]
  apply Proves.andI
  · apply Proves.andE2 (A := P)
    apply Proves.hyp
    simp                      -- P ∧ Q ∈ [P ∧ Q]
  · apply Proves.andE1 (B := Q)
    apply Proves.hyp
    simp
```

`apply Proves.impI` turns the goal `Proves [] (P ∧ Q → Q ∧ P)` into its premise `Proves [P ∧ Q] (Q ∧ P)`. After `apply Proves.andI`, there are two goals. For the first, `apply Proves.andE2 (A := P)` says "this Q comes from a conjunction $P \wedge Q$": we must name the left conjunct `P` ourselves, because ∧E2 has a formula in its premise that does not occur in its conclusion, and Lean cannot guess it. That is a general feature of elimination rules, and it is why searching for natural-deduction proofs is harder than searching in the sequent calculus. Finally `apply Proves.hyp` leaves the membership goal `P ∧ Q ∈ [P ∧ Q]`, which `simp` proves by simplifying it to `True`.

Truth tables in Lean

The truth-table semantics is a recursive function into `Bool`:

```
/-- The truth-table value of a formula, given a value for every atom. -/
def eval (v : Nat → Bool) : Formula → Bool
  | .var n   => v n
  | .top     => true
  | .bot     => false
  | .and A B => eval v A && eval v B
  | .or A B  => eval v A || eval v B
  | .imp A B => !eval v A || eval v B
```

A valuation `v : Nat → Bool` gives a truth value to every atom. To check a formula with n atoms we need the 2^n rows of its table:

```
/-- All valuations of the atoms `0, ..., n - 1` (the other atoms are `false`). -/
def valuations : Nat → List (Nat → Bool)
  | 0     => [fun _ => false]
  | n + 1 => (valuations n).flatMap fun v =>
      [v, fun k => if k = n then true else v k]

/-- A formula whose atoms are among `0, ..., n - 1` is a tautology when every
```

```
row of its truth table is `true`. -/
def isTautology (n : Nat) (A : Formula) : Bool :=
  (valuations n).all fun v => eval v A
```

```
/-- info: true -/
#guard_msgs in
#eval isTautology 2 (.imp (.imp P Q) (.imp (.or P R) (.or Q R)))

/-- info: false -/
#guard_msgs in
#eval isTautology 2 (.imp (.or P Q) P)

/-- info: true -/
#guard_msgs in
#eval isTautology 1 (.or P (.neg P))
```

(`flatMap` maps each valuation to a list and concatenates the lists, like Rust's `flat_map`; each existing row is kept and copied with atom `n` set to `true`.)

Soundness

Now the theorem. It says: if `Γ ⊢ A` is derivable, then every valuation that makes all of `Γ` true makes `A` true. The proof is by **induction on the derivation**, one case per rule, the same kind of proof as the soundness of the `Arith` checker in Chapter 3:

```
/-- **Soundness.** If `Γ ⊢ A` is derivable, then every valuation that makes all
of `Γ` true makes `A` true. By induction on the derivation: one case per rule. -/
theorem soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ v : Nat → Bool, (∀ B ∈ Γ, eval v B = true) → eval v A = true := by
  induction h with
  | hyp hmem =>
    -- The conclusion is one of the assumptions, which are all true.
    intro v hΓ
    exact hΓ _ hmem
  | topI =>
    intro v _
    rfl
  | botE _ ih =>
    -- The premise would make `false = true`: this case cannot happen.
    intro v hΓ
    have hbot := ih v hΓ
    simp [eval] at hbot
  | andI _ _ ih₁ ih₂ =>
    intro v hΓ
    simp [eval, ih₁ v hΓ, ih₂ v hΓ]
  | andE1 _ ih =>
    intro v hΓ
    have hand := ih v hΓ
    simp [eval] at hand
    exact hand.1
  | andE2 _ ih =>
    intro v hΓ
    have hand := ih v hΓ
    simp [eval] at hand
    exact hand.2
  | orI1 _ ih =>
    intro v hΓ
    simp [eval, ih v hΓ]
  | orI2 _ ih =>
    intro v hΓ
    simp [eval, ih v hΓ]
  | orE _ _ _ ih ihA ihB =>
    -- The disjunction is true, so one disjunct is; use the matching branch,
    -- whose context is Γ extended with that (true) disjunct.
    intro v hΓ
```

```
    have hor := ih v hΓ
    simp [eval] at hor
    rcases hor with hA | hB
    · apply ihA v
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hA
      · exact hΓ C hC
    · apply ihB v
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hB
      · exact hΓ C hC
  | @impI Γ A B _ ih =>
    -- If A is false the implication is true; if A is true, the premise
    -- (with A added to the context) gives B.
    intro v hΓ
    cases hA : eval v A with
    | false => simp [eval, hA]
    | true =>
      have hB : eval v B = true := by
        apply ih v
        intro C hC
        rcases List.mem_cons.mp hC with rfl | hC
        · exact hA
        · exact hΓ C hC
      simp [eval, hB]
  | impE _ _ ih₁ ih₂ =>
    intro v hΓ
    have himp := ih₁ v hΓ
    have hA := ih₂ v hΓ
    simp [eval, hA] at himp
    exact himp
```

`induction h with` produces one goal per constructor of `Proves`, and in each case the induction hypotheses (`ih`, `ih₁`, ...) state the theorem for the premises. Let us go through the cases.

- **hyp**: the conclusion is in the context, and every formula in the context is true by assumption `hΓ`.
- **topI**: `eval v .top` is `true` by definition, so `rfl` closes the goal.
- **botE**: the induction hypothesis says `eval v .bot = true`; `simp [eval] at hbot` rewrites it to `false = true`, recognizes it as impossible and closes the goal. The premise of ⊥E can never be true, so its conclusion is never at risk.
- **andI, orI1, orI2**: `simp` unfolds `eval` and uses the induction hypotheses to finish the Boolean computation.
- **andE1, andE2**: the hypothesis says `eval v A && eval v B = true`; `simp` turns it into a conjunction of two equations, and we take the one we need.
- **orE**: the main hypothesis says one of the disjuncts is true. In each case we apply the induction hypothesis for the corresponding branch, which needs every formula in the *extended* context to be true: the new head is the true disjunct, and the rest is `Γ`. The `rcases ... with rfl | hC` splits membership in `A :: Γ` into "it is `A`" and "it is in `Γ`".
- **impI**: the conclusion is `!eval v A || eval v B`. If `A` is false, this is true already. If `A` is true, the context `A :: Γ` is all true, so the induction hypothesis gives `eval v B = true`. The syntax `@impI Γ A B _ ih` names the implicit arguments so that we can talk about `A` and `B`.
- **impE**: both premises are true, and `simp` combines them.

There is no clever idea in this proof, and that is the point: soundness is checked rule by rule, and a rule like tonk would leave a case that no tactic can close.

What soundness buys

Soundness turns a semantic fact into a statement about proofs. The first consequence is consistency, this time with a complete proof:

```
/-- **Consistency.** `⊥` cannot be derived from no assumptions: under any
valuation, `eval v ⊥ = false`. -/
theorem consistency : ¬ Proves [] .bot := by
  intro h
  have := soundness h (fun _ => false) (by simp)
  simp [eval] at this
```

A derivation of `⊥` would make `eval v .bot` true for any valuation, but it is `false`. The same argument shows that specific formulas have no derivation, something no amount of failed searching could establish:

```
/-- An atom is not derivable: the valuation that makes every atom false is a
counterexample. -/
theorem atom_not_derivable : ¬ Proves [] P := by
  intro h
  have := soundness h (fun _ => false) (by simp)
  simp [eval, P] at this

/-- `(P ∨ Q) → P` is not derivable: take `P` false and `Q` true. -/
theorem or_imp_not_derivable : ¬ Proves [] (.imp (.or P Q) P) := by
  intro h
  have := soundness h (fun n => n = 1) (by simp)
  simp [eval, P, Q] at this
```

(`fun n => n = 1` is a valuation: the proposition `n = 1` is decidable, so Lean turns it into the `Bool` that is `true` exactly for atom 1, which is `Q`.)

Soundness has a limit, and it is a deep one. The converse, **completeness**, would say that every formula true in all rows is derivable. For the rules of this chapter it is false:

```
/-- Excluded middle is a tautology: true in every row of its truth table. So
soundness alone cannot show that it is *not* derivable. (Chapter 5 shows it
with a Kripke model.) -/
theorem lem_tautology (v : Nat → Bool) : eval v (.or P (.neg P)) = true := by
  cases h : v 0 <;> simp [eval, Formula.neg, P, h]
```

Excluded middle, $A \vee \neg A$, is a tautology, yet it has no derivation in our system. Truth tables cannot show this, because it is true in every row. Chapter 5 builds two finer semantics, Kripke models and Heyting algebras, and uses them to prove in Lean that `Proves [] (.or P P.neg)` is impossible.

4.9 Classical logic, in one rule

The system of this chapter is **intuitionistic** natural deduction. Classical logic, the logic of truth tables, is obtained by adding one more rule, for instance *reductio ad absurdum*:

$$\frac{\Gamma, \neg A \vdash \bot}{\Gamma \vdash A}\ \mathsf{RAA}$$

"To prove A, assume it is false and derive a contradiction." With RAA, $A \vee \neg A$ becomes derivable, and the system is complete for truth tables. RAA is neither an introduction nor an elimination: it belongs to no connective and falls outside the harmony story. That is not an argument against classical logic, but it is a first sign that the two systems differ in character. Chapter 5 is devoted to the difference, and to why Lean's core logic is intuitionistic while `Classical.em` sits in its library.

4.10 Putting it together: one formula, four checks

Let us follow a single formula through everything in this chapter:

$$(A \to B) \to A \vee C \to B \vee C$$

"If A implies B, then A or C implies B or C." In programming terms it is `map` for the left side of a sum.

On paper. Work bottom-up. Two $\to$I steps assume $A \to B$ (call it g) and $A \vee C$ (call it d), leaving the goal $B \vee C$. Split d with $\vee$E. In the case A (call it a), apply g to a with $\to$E and finish with $\vee$I1; in the case C, use $\vee$I2. With $\Gamma = A \to B, A \vee C$:

$$\dfrac{\Gamma \vdash A \vee C \qquad \dfrac{\dfrac{\Gamma, A \vdash A \to B \qquad \Gamma, A \vdash A}{\Gamma, A \vdash B}\,{\to}\mathsf{E}}{\Gamma, A \vdash B \vee C}\,{\vee}\mathsf{I}_1 \qquad \dfrac{\Gamma, C \vdash C}{\Gamma, C \vdash B \vee C}\,{\vee}\mathsf{I}_2}{\Gamma \vdash B \vee C}\,{\vee}\mathsf{E}$$

followed by two $\to$I steps, with the leaves closed by Hyp.

In Lean, shallowly. The tactic proof follows the paper proof step by step:

```
/-- The walkthrough formula of the chapter, `(A → B) → A ∨ C → B ∨ C`. -/
theorem or_map (p q r : Prop) : (p → q) → p ∨ r → q ∨ r := by
  intro hpq h              -- →I twice
  cases h with             -- ∨E on h : p ∨ r
  | inl hp =>
    left                   -- ∨I1
    exact hpq hp           -- →E
  | inr hr =>
    right                  -- ∨I2
    exact hr
```

In Lean, deeply. The same derivation as a value of `Proves`. The eliminated formulas must be given explicitly for $\vee$E (which disjunction we split) and $\to$E (which antecedent we prove):

```
/-- `⊢ (P → Q) → (P ∨ R) → (Q ∨ R)`: two →I, one ∨E, two ∨I, one →E. -/
theorem or_map_derivation : Proves [] (.imp (.imp P Q) (.imp (.or P R) (.or Q R))) := by
  apply Proves.impI                    -- assume P → Q
  apply Proves.impI                    -- assume P ∨ R
  apply Proves.orE (A := P) (B := R)   -- case analysis on P ∨ R
  · exact Proves.hyp (by simp)         -- the disjunction is an assumption
  · apply Proves.orI1                  -- case P: prove Q, then Q ∨ R
    apply Proves.impE (A := P)
    · exact Proves.hyp (by simp)       -- P → Q
    · exact Proves.hyp (by simp)       -- P
  · apply Proves.orI2                  -- case R: prove R, then Q ∨ R
    exact Proves.hyp (by simp)
```

In Rust. The proof tree for the checker:

```
// ⊢ (A → B) → A ∨ C → B ∨ C: a case analysis (∨E) on the disjunction `d`.
let or_map = imp_i(
```

```
    "g",
    f("A → B"),
    imp_i(
        "d",
        f("A ∨ C"),
        or_e(
            hyp("d"),
            "a", // case A: apply g, then choose the left side
            or_i1(imp_e(hyp("g"), hyp("a")), f("C")),
            "c", // case C: choose the right side
            or_i2(f("B"), hyp("c")),
        ),
    ),
);
let d2 = checker
    .prove(&[], &f("(A → B) → A ∨ C → B ∨ C"), &or_map)
    .expect("a correct proof");
print!("{d2}");
```

```
⊢ (A → B) → A ∨ C → B ∨ C                →I g
└─ A → B ⊢ A ∨ C → B ∨ C                 →I d
   └─ A → B, A ∨ C ⊢ B ∨ C                ∨E a c
      ├─ A → B, A ∨ C ⊢ A ∨ C             Hyp d
      ├─ A → B, A ∨ C, A ⊢ B ∨ C          ∨I1
      │  └─ A → B, A ∨ C, A ⊢ B           →E
      │     ├─ A → B, A ∨ C, A ⊢ A → B    Hyp g
      │     └─ A → B, A ∨ C, A ⊢ A        Hyp a
      └─ A → B, A ∨ C, C ⊢ B ∨ C          ∨I2
         └─ A → B, A ∨ C, C ⊢ C           Hyp c
```

The printed derivation is the paper tree, upside down and with every context written out: ∨E discharges `a` in its second premise and `c` in its third, and the contexts grow accordingly. Four representations, one proof; `#print or_map` would show a fifth, a program.

4.11 Common confusions

"**⊢ and → are the same thing.**" They are interderivable, by →I and →E, but not the same. $\Gamma \vdash A$ is a judgment about A, made in the metalanguage; $A \to B$ is a formula, an object the judgment talks about. The distinction is what lets Chapter 6 change the rules for contexts without touching any connective.

"**A tautology must be provable.**" For classical logic, yes: that is completeness. For the intuitionistic rules of this chapter, no: $A \vee \neg A$ is a tautology without a derivation. Soundness goes one way only, and Chapter 5 gives the semantics for which intuitionistic logic is complete.

"**`exfalso`, `contradiction` and ⊥E are classical reasoning.**" They are not. ⊥E is an ordinary intuitionistic rule: it uses a proof of ⊥ that you already have. Classical reasoning is RAA, which *produces* a proof of A from a refutation of $\neg A$. The difference between "from a contradiction, anything" and "if $\neg A$ is contradictory, then A" is the subject of Chapter 5.

"**Harmony is about truth.**" Harmony is a property of rules and proofs, checked without any semantics: can detours be removed, can formulas be rebuilt? That it also guarantees consistency (through normalization) is a theorem, not the definition.

"**Tactic proofs are a different kind of proof.**" A tactic script is a program that builds a proof term, and `#print` shows the term. The kernel never sees the tactics.

4.12 Summary

- A proposition is syntax; a judgment such as "A true" or $\Gamma \vdash A$ is an assertion about it. Inference rules relate judgments.
- Natural deduction gives each connective introduction rules (how to prove it) and elimination rules (how to use it). $\to$I and $\vee$E discharge assumptions; negation is $A \to \bot$ and inherits the rules of $\to$.
- Tree notation (with discharge labels) and sequent notation (with explicit contexts) carry the same information.
- Harmony: eliminations take out exactly what introductions put in. Local soundness is β-reduction, local completeness is η-expansion, and the shape of each elimination follows from its introductions. Prior's `tonk` fails local soundness and proves $\bot$.
- Normal derivations have the subformula property; normalization implies consistency. The sequent calculus with cut elimination is the same logic organized for proof search.
- In Lean, propositions are types in `Prop` and proofs are terms; tactics apply rules backwards, and `#print` shows the resulting term. Inductive propositions get harmonious eliminators for free.
- A deep embedding makes formulas and derivations data. We proved soundness for truth tables by induction on derivations, and derived consistency and non-derivability results from it. Completeness fails: excluded middle is a tautology without a derivation.
- The Rust crate `natded` checks proof trees, explains its rejections, tests soundness against truth tables and normalizes proofs; with `tonk` enabled it accepts a proof of $\bot$.

4.13 Exercises

1. Derive $\vdash A \to (B \to A)$ and $\vdash (A \to B \to C) \to (A \wedge B \to C)$ in tree notation with discharge labels. In the first, one assumption is discharged but never used: which? Then check both proofs with `natded`. *Hint:* the unused assumption is discharged by a $\to$I whose label appears on no leaf. Chapter 6 calls this weakening.
2. State local soundness and local completeness for $\top$. One of them is vacuous: which, and why? Write the expansion for $\top$ as a Lean equation on `Unit` and prove it. *Hint:* $\top$ has an introduction and no elimination, so no detour can form. The equation is in `Harmony.lean`.
3. In Lean, using only the tactics of this chapter, prove `p ∨ (q ∧ r) → (p ∨ q) ∧ (p ∨ r)`, `¬(p ∨ q) → ¬p ∧ ¬q` and `(p → q) → (p → r) → p → q ∧ r`. Then `#print` one of them and match the term against your tactic script. *Hint:* `rintro (hp | ⟨hq, hr⟩)` splits the first hypothesis; for the second, `intro h` and then `constructor`, and in each branch `intro` and `apply h`.
4. In the deep embedding, build a value of `Proves [] (.imp P P.neg.neg)` (the derivation of $A \to \neg\neg A$ from this chapter). Then prove `¬ Proves [] (.imp (.imp P Q) (.imp Q P))` using `soundness`. *Hint:* unfold `Formula.neg` mentally: you need two `impI` and one `impE`. For the second part,

find a row of the truth table that falsifies the formula and pass it to `soundness` as in `or_imp_not_derivable`.

5. Add the rule $\dfrac{\Gamma \vdash \top}{\Gamma \vdash C}$ ("$\top$E") to a copy of the `natded` checker and derive $\vdash \bot$. Which harmony condition does the rule violate? Compare it with `tonk`. *Hint:* $\top$I puts nothing in, so this elimination takes out more than was put in: the detour $\top$I/$\top$E has nothing to reduce to.
6. Consider a "conjunction" $A \circ B$ with the introduction rule of $\wedge$ and only the elimination $\wedge$E1. Show that it is locally sound but not locally complete, that adding it does not make $\bot$ derivable, and say what is lost. *Hint:* read $A \circ B$ as $A \wedge B$ to translate every derivation, so consistency is inherited; but B can never be recovered from $A \circ B$.